

Structuring LLM Arguments: A Logic of Argumentation Based Approach

Aaron Habyarimana

July 2, 2026
Version: CleanThesis 0.4.1 - JGU patches



Johannes Gutenberg University Mainz
FB08
Institute of Computer Science
Data Mining

Bachelorarbeit

Structuring LLM Arguments: A Logic of Argumentation Based Approach

Aaron Habyarimana

- | | |
|--------------------|--|
| <i>1. Reviewer</i> | Prof. Dr. Stefan Kramer
Institute of Computer Science - Data Mining
Johannes Gutenberg University Mainz |
| <i>2. Reviewer</i> | Maike Paetzel-Prüsmann
Institute of Computer Science
Johannes Gutenberg University Mainz |
| <i>Supervisors</i> | Derian Boer and Lennart Baur |

July 2, 2026

Aaron Habyarimana

Structuring LLM Arguments: A Logic of Argumentation Based Approach

Bachelorarbeit, July 2, 2026

Reviewers: Prof. Dr. Stefan Kramer and Maike Paetzel-Prüsmann

Supervisors: Derian Boer and Lennart Baur

Johannes Gutenberg University Mainz

Data Mining

Institute of Computer Science

FB08

Staudingerweg 9

55128 Mainz

Abstract

This thesis presents an end-to-end pipeline that generates, structures, and formally evaluates Abstract Argumentation Frameworks from natural-language debate topics using large language models. Rather than treating attack detection as a binary decision, the system prompts a language model to produce categorized attack candidates, each annotated with an attack type, namely rebuttal, undercutting, or undermining, a confidence score, and a natural-language justification. A two-stage confidence filter routes candidates to acceptance, rejection, or verification. Accepted attacks are reduced to binary facts and handed to a Prolog solver that computes grounded, preferred, and stable extensions under Dung semantics. Evaluated on eight topics from the UKP Sentential Argument Mining Corpus, every framework is cyclic and six of eight admit several preferred and stable extensions, each corresponding to an internally coherent position that a standalone language model leaves implicit. Beyond this cross-topic analysis, the thesis sets up an ablation that isolates the effect of typed against binary prompting and a gold-standard evaluation of the extraction quality.

Abstract (german)

Diese Arbeit stellt eine durchgängige Pipeline vor, die mithilfe großer Sprachmodelle aus natürlichsprachlichen Debattenthemen abstrakte Argumentationsframeworks erzeugt, strukturiert und formal auswertet. Statt die Angriffserkennung als binaere Entscheidung zu behandeln, erzeugt das System kategorisierte Angriffskandidaten, die jeweils mit einem Angriffstyp (Rebuttal, Undercutting oder Undermining), einem Konfidenzwert und einer Begründung versehen sind. Ein zweistufiger Konfidenzfilter leitet die Kandidaten zur Annahme, Ablehnung oder Verifikation. Angenommene Angriffe werden auf binäre Fakten reduziert und an einen Prolog-Solver übergeben, der grounded, preferred und stable Extensions unter Dung-Semantik berechnet. In der Auswertung auf acht Themen des UKP-Korpus ist jedes Framework zyklisch, und sechs von acht Themen besitzen mehrere preferred und stable Extensions, die jeweils einer in sich konsistenten Position entsprechen. Ergänzend bereitet die Arbeit eine Ablation zum Vergleich von typisiertem und binärem Prompting vor.

Contents

1. Introduction	1
1.1. Motivation	1
1.2. Problem Statement and Research Questions	1
1.3. Contributions	2
1.4. Thesis Structure	3
2. Foundations	5
2.1. Abstract Argumentation Frameworks	5
2.2. Attack Typology	6
2.3. Argument Mining	7
2.4. Large Language Models	7
3. Related Work	9
3.1. LLM-based End-to-End Argumentation Pipelines	9
3.2. LLMs for Attack Relation Extraction	10
3.3. Argument Mining on the UKP Corpus	10
3.4. Related Formalisms	10
3.5. Positioning	11
4. Method: The Pipeline	13
4.1. Overview	13
4.2. Step 1: Argument Acquisition and Atomization	13
4.3. Step 2: Attack Detection and Classification	14
4.4. Step 3: Confidence Filtering and Verification	15
4.5. Step 4: AAF Conversion	15
4.6. Step 5: Solving under Dung Semantics	16
5. Evaluation	17
5.1. Experimental Setup	17
5.2. Cross-Topic Results	17
5.3. Typed versus Binary Prompting	18
5.4. Attack Quality against a Gold Standard	19
5.5. Case Study: Nuclear Energy	19
5.6. Error Analysis	20
6. Discussion	21
6.1. What the Formal Layer Adds	21
6.2. Threats to Validity	21
6.3. Design Trade-offs	21

7. Conclusion	23
7.1. Future Work	23
Bibliography	25
List of Figures	27
List of Tables	29
List of Listings	31
A. Example Appendix	33
A.1. Appendix Section 1	33
A.2. Appendix Section 2	33

Introduction

1.1 Motivation

Human argumentation is hard to capture by machines because it does not follow the rules of classical logic. Instead of deriving truth from fixed premises, a debate consists of arguments and counter-arguments, and evidence enters weighted or by comparison with other evidence rather than as a pure fact that is true or false. This resembles a verdict that rests on aggravating and mitigating circumstances rather than on a single decisive proof. The problem is not narrow or merely technical. It connects to deeper questions such as algorithmic fairness, the reasoning behaviour of neural models, and the hallucinations of language models.

One established way to formalise arguments is the logic of argumentation [8], which provides a language for representing the structure of reasons that support or undercut a line of reasoning. While this approach is useful and valid, several open questions remain. In particular, it is not clear where arguments and counter-arguments come from in practical applications and how they are in fact related. The examples in the literature often appear constructed, and providing the arguments frequently seems harder than the reasoning over them.

Large language models (LLMs) sit at the opposite end. They can produce reasons for or against a cause in fluent natural language, but they lack the formal structure that would allow any systematic analysis of how those reasons relate. A standalone LLM can be asked to summarise a debate, yet its answer is another ungrounded generation without a guarantee of internal consistency. The proposal of this thesis is to use the LLM for what it is good at, namely producing arguments and identifying their attacks, and to hand the resulting structure to a formal layer that is good at reasoning about consistency. The formal layer reconstructs deeply layered argumentations and makes explicit which sets of arguments can be jointly accepted, a property that the raw LLM output leaves implicit.

1.2 Problem Statement and Research Questions

This thesis builds and evaluates an end-to-end pipeline that turns natural-language debate text into a formally evaluated Abstract Argumentation Framework (AAF) in the sense of Dung [4]. An LLM proposes categorized attack

candidates between arguments, a confidence filter routes them to acceptance, rejection, or verification, and a Prolog solver computes grounded, preferred, and stable extensions over the accepted attacks.

The work is guided by four research questions.

RQ1 (Feasibility). Can an end-to-end pipeline be built that turns natural-language arguments into formal Dung extensions through LLM-based attack extraction and a symbolic solver, and what do the resulting frameworks look like across a range of debate topics?

RQ2 (Typed vs. binary extraction). Does *typed* attack prompting, which forces the model to name the attack type and justify it, produce different or more reliable attack relations than plain *binary* prompting, and how does this affect the resulting frameworks?

RQ3 (Extraction quality). How well do the extracted attacks match a manual gold annotation in terms of precision, recall, and F1, and which error types dominate?

RQ4 (Argumentative structure). What argumentative structures do the formal semantics reveal across the eight UKP topics, in particular regarding cycles and multiple extensions, and when does the formal layer add information beyond the grounded extension?

1.3 Contributions

The thesis makes the following contributions.

- A complete and reproducible pipeline that connects LLM-based argument mining with Dung’s abstract argumentation, from atomization through categorized attack extraction and confidence filtering to a Prolog solver.
- An empirical analysis of the resulting frameworks over eight topics of the UKP Sentential Argument Mining Corpus [14], reporting framework size, cycles, and the number of grounded, preferred, and stable extensions (RQ1, RQ4).
- An ablation that compares typed against binary attack prompting on an identical argument set, isolating the effect of the categorization that the underlying paper advertises but does not measure (RQ2).
- A small manually annotated gold standard that yields precision, recall, and F1 for the attack detection on one topic (RQ3).

A condensed version of the pipeline and the cross-topic evaluation has been submitted to IJCLR. This thesis extends that work with explicit research questions, an expanded background, and the two additional experiments for RQ2 and RQ3.

1.4 Thesis Structure

Chapter 2 introduces the formal background, namely Dung’s abstract argumentation frameworks and their semantics, the attack typology of Pollock and ASPIC+, and the basics of argument mining and language models.

Chapter 3 reviews related work on LLM-based argumentation pipelines and positions the thesis against it.

Chapter 4 describes the pipeline in detail, step by step, together with the design decisions behind it.

Chapter 5 presents the experimental setup and results, including the cross-topic evaluation, the typed-versus-binary ablation, the gold-standard evaluation, and a detailed case study.

Chapter 6 discusses what the formal layer adds, the threats to validity, and the main design trade-offs.

Chapter 7 summarises the answers to the research questions and outlines future work.

Foundations

This chapter introduces the formal and conceptual background the thesis builds on. Section 2.1 defines Dung’s abstract argumentation frameworks and the three semantics the pipeline computes. Section 2.2 introduces the attack typology of Pollock and ASPIC+ that the extraction step uses. Section 2.3 sketches argument mining, and Section 2.4 covers the aspects of large language models relevant to this work.

2.1 Abstract Argumentation Frameworks

Abstract Argumentation Frameworks (AAFs), introduced by Phan Minh Dung in 1995 [4], are the most established formal model of argumentation. The framework is deliberately abstract. It treats arguments as atomic entities without internal structure and represents only the conflicts between them.

Definition 2.1 (Argumentation Framework). An argumentation framework is a pair $AF = \langle \mathcal{A}, \mathcal{R} \rangle$, where $\mathcal{A}$ is a finite set of arguments and $\mathcal{R} \subseteq \mathcal{A} \times \mathcal{A}$ is a binary attack relation. We say that an argument a attacks an argument b if $(a, b) \in \mathcal{R}$.

The central question is which sets of arguments can rationally be accepted together. The following notions, all due to Dung [4], build up to an answer.

Definition 2.2 (Conflict-freeness and defense). A set $S \subseteq \mathcal{A}$ is *conflict-free* if there are no arguments $a, b \in S$ with $(a, b) \in \mathcal{R}$. An argument a is *defended* by S if for every b with $(b, a) \in \mathcal{R}$ there is some $c \in S$ with $(c, b) \in \mathcal{R}$.

Definition 2.3 (Admissibility and characteristic function). A conflict-free set S is *admissible* if it defends every one of its elements. The characteristic function $F_{AF} : 2^{\mathcal{A}} \rightarrow 2^{\mathcal{A}}$ maps a set to the arguments it defends, $F_{AF}(S) = \{a \in \mathcal{A} \mid S \text{ defends } a\}$.

On top of admissibility, Dung defines several *semantics*, each formalising a different standard of collective acceptability. This thesis computes three of them, chosen to span a wide spectrum of strictness.

- The **grounded extension** is the least fixed point of F_{AF} , obtained by iterating F_{AF} from the empty set. It is the most cautious semantics, it always exists, and it is unique for finite frameworks.

Placeholder: worked example of a small framework with several preferred extensions.

Fig. 2.1.: Running example of a small argumentation framework. To be replaced with a TikZ figure and a step-by-step computation of the grounded, preferred, and stable extensions.

- A **preferred extension** is a maximal admissible set. It represents the largest internally coherent position a debate supports. A framework can have several preferred extensions.
- A **stable extension** is a conflict-free set S that attacks every argument outside S . Stable extensions are the strictest of the three and are not guaranteed to exist.

The grounded semantics collapses a debate to a single cautious set, whereas preferred and stable extensions, which may be multiple, reveal whether a topic supports several internally consistent positions. This distinction is central to the evaluation in Chapter 5.

2.2 Attack Typology

Dung's framework is abstract in that it does not say *why* one argument attacks another. For the extraction step, however, the kind of attack matters. Pollock [13] distinguishes three attack types in natural-language debate, a distinction also used in the ASPIC+ framework [11].

- A **rebuttal** contradicts the *conclusion* of the target. An example is the pair “nuclear power is clean” against “nuclear plants cause catastrophic contamination”.
- An **undercutting** attack challenges the *inference* from premises to conclusion. The conclusion does not follow even if the premises hold, for example when low CO₂ emissions are taken to imply the absence of any long-term risk.

- An **undermining** attack hits a *premise* the target relies on, without engaging the conclusion directly. The claim “containment makes plants safe” is undermined by “radiation cannot be contained indefinitely”.

A property that matters for the construction of the framework is that a rebuttal is inherently mutual. If two conclusions negate the same proposition, then one is true only if the other is false, so the conflict holds in both directions. Without a preference or strength ordering, which Dung’s abstract framework does not provide, neither side takes precedence. The pipeline therefore symmetrizes rebuttals, in line with the treatment of rebuttals in ASPIC+ [11]. Undercutting and undermining attacks, by contrast, remain directed. This asymmetry has consequences for the cycles that appear in the resulting frameworks, which Chapter 5 discusses.

2.3 Argument Mining

Argument mining (AM) aims to automatically identify argumentative components, such as claims and premises, and the relations between them from natural-language text. Traditional approaches decompose the problem into subtasks including argument detection, stance classification, and relation prediction, and address them with supervised learning on annotated corpora [9]. More recently, LLMs have begun to replace task-specific classifiers with prompt-based methods that perform extraction, relation identification, stance assessment, summarization, and generation in a single framework [10]. Attack relation extraction in particular benefits from this shift, since LLMs can use contextual understanding to identify subtle argumentative relations that surface-level classifiers miss. Most existing work, however, evaluates AM subtasks in isolation and stops before connecting the extracted relations to formal reasoning, which leaves the question of which arguments are collectively acceptable unanswered. This thesis targets exactly that gap.

2.4 Large Language Models

Large language models are neural networks trained on large text corpora to predict the next token, and through that objective they acquire a broad ability to follow instructions given in natural language. Only a few properties matter for this thesis. First, model behaviour is steered through *prompts*, so the wording of the task, including whether the model is asked to classify an attack type, directly shapes the output. Second, the sampling *temperature* controls randomness, and a temperature of zero makes the output as deterministic as the model allows, which the pipeline uses for reproducibility. Determinism is not absolute, since server-side execution can still introduce small variations. Third, the pipeline uses an open-weight model, which keeps the experiments

reproducible and independent of a proprietary service. The concrete model and parameters are given in Section 5.1.

Related Work

This chapter reviews work that combines language models with formal or relational argumentation and positions the present thesis against it. It first discusses end-to-end pipelines that turn text into argumentation frameworks, then work that uses LLMs specifically for relation extraction, the use of the UKP corpus, and finally related formalisms.

3.1 LLM-based End-to-End Argumentation Pipelines

The closest work to this thesis is ARGUS [2], which combines LLM-based argument mining with the construction of argumentation frameworks and symbolic solvers in an end-to-end architecture. ARGUS shares the central idea pursued here, namely to let an LLM handle the noisy natural-language part of the problem and to delegate the acceptability question to a formal solver. It sets out the architecture and the Dung-semantics perspective but does not report an empirical evaluation of the resulting frameworks, so it leaves open how many attacks such a pipeline accepts, whether the resulting frameworks are cyclic, and how many extensions they admit. The present work adopts this end-to-end perspective and supplies exactly that missing empirical layer, a reproducible implementation with logged runs and a cross-topic analysis over eight topics.

ArgLLMs [5] also augments LLMs with formal argumentative reasoning but takes a quantitative route. Instead of asking which arguments can be jointly accepted, it builds Quantitative Bipolar Argumentation Frameworks (QBAFs) in which each argument carries a numerical strength that gradual semantics propagate along support and attack edges. The system is evaluated on binary claim verification, where the aggregated strength of the arguments for a claim is turned into an accept or reject decision. This differs from the present thesis in two respects. The output is a single scalar per claim rather than a set of jointly admissible arguments, and the evaluation targets a downstream verification task rather than the structure of the argumentation itself.

A further recent example of an end-to-end LLM system for argument mining is the work of Pirozelli et al. [12], which applies LLMs to the argument mining subtasks within a single system rather than through separate task-specific classifiers. It reflects the broader trend that LLMs are becoming the default engine for extraction, but like most argument mining work it stops at the extracted structure and does not connect the result to Dung semantics or

evaluate the collective acceptability of the mined arguments. The pipeline in this thesis differs from all three systems in its use of categorized attack candidates with explicit justifications and confidence-based filtering, and in computing set-based Dung extensions rather than numerical strengths or raw extracted relations.

3.2 LLMs for Attack Relation Extraction

The extraction step of the pipeline, which classifies the relation between every pair of atomic arguments, is most directly related to work that studies LLMs on relation-based argument mining. Gorur et al. [6] ask precisely whether large language models can identify the relations, such as support and attack, that hold between arguments, the subtask on which the present pipeline depends. Their perspective is diagnostic, in that they measure how well LLMs recover these relations in isolation, whereas the present thesis treats relation extraction as one stage of a larger pipeline and feeds its output into a formal solver. Two design choices set the extraction here apart. First, the LLM is asked not only whether an attack exists but also which of the three Pollock types [13] it instantiates, which forces engagement with the argumentative structure rather than a surface judgment on opposing stances. Second, each decision carries a confidence score that routes borderline cases to a separate verification prompt instead of accepting them outright. The value of the typed prompt over a plain binary one is measured directly in the ablation of Section 5.3.

3.3 Argument Mining on the UKP Corpus

The evaluation uses the UKP Sentential Argument Mining Corpus [14], which provides sentence-level stance annotations for eight controversial topics. Prior work of the supervising group applies knowledge-based graph argument mining with topic modeling to the same corpus [7] and reports an F1 around 67 percent. That number serves as a reference point for the gold-standard evaluation in Section 5.4.

3.4 Related Formalisms

Most closely related on the formal side is the recent work of Alfano et al. [1], which connects LLM-based argument mining with argumentation and description logics through a FAKB and QBAF framework. Their approach enriches

the formal layer, both by adding logical structure through description logics and by retaining numerical strengths through the QBAF, which makes it more expressive but also heavier, since it commits to a strength ordering that has to be justified. The present thesis deliberately stays with Dung’s abstract framework instead. The reason is that the question of interest here is set-based, namely which arguments can be jointly accepted, rather than numerical, namely how strong each argument is. Without a preference or strength ordering, binary attack facts are the natural representation for that question, as Chapter 4 argues in more detail. The typed attacks and confidence scores are not discarded but retained as metadata, which keeps the door open to a richer formalism such as ASPIC+ [11] or a QBAF as future work.

3.5 Positioning

In short, this thesis takes the end-to-end perspective of ARGUS and of the LLM argument mining systems above, grounds it in a concrete and reproducible implementation, and evaluates the resulting frameworks empirically, which the closest prior systems leave undone. It differs from the gradual and logic-augmented approaches of ArgLLMs and Alfano et al. by computing classical Dung extensions rather than numerical strengths, and it goes beyond diagnostic relation-extraction studies such as Gorur et al. by connecting the extracted attacks to a formal solver. Its methodological contribution, a typed attack extraction with confidence-based filtering, is measured directly in the ablation of Section 5.3.

Method: The Pipeline

This chapter describes the pipeline that turns raw debate text into a formally evaluated Dung framework. Section 4.1 gives an overview, and the remaining sections describe the five steps in turn, each with the design decision behind it.

4.1 Overview

The pipeline runs in five steps. Step one acquires arguments and atomizes them into self-contained claims. Step two examines every pair of claims and proposes categorized attack candidates. Step three filters those candidates by confidence and verifies the borderline cases. Step four converts the accepted candidates into binary Dung attack facts. Step five solves the resulting framework under grounded, preferred, and stable semantics with a Prolog solver.

4.2 Step 1: Argument Acquisition and Atomization

Arguments enter the pipeline in one of three ways. They are generated from scratch by prompting an LLM to produce n pro and n con arguments on a topic, extracted from an existing text corpus, or loaded directly from an argument database. In all cases an argument is represented by a tuple of an identifier, a stance label, and the argument text.

Natural-language arguments frequently bundle several claims. A compound sentence such as “Although nuclear energy produces waste, it is cleaner than coal” mixes a concession with a main claim, and treating it as one unit would distort the attack relations extracted next. The pipeline therefore prompts an LLM to split each argument into self-contained atomic claims under three rules.

1. **Drop concessive fragments.** Every component introduced by “although”, “while”, “despite”, and similar markers is a rhetorical hedge rather than a standalone argument and is dropped.
2. **Resolve pronouns.** A pronoun that clearly refers to a specific subject is replaced by that subject. A fragment whose referent cannot be recovered is removed, so that every atom is understandable in isolation.

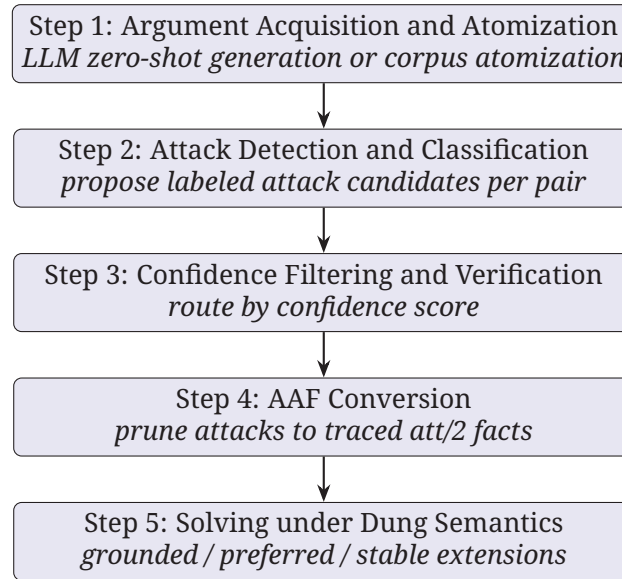


Fig. 4.1.: Overview of the pipeline, from raw debate text to formally evaluated Dung extensions in five steps.

3. **Filter non-argumentative content.** A statement that neither supports nor opposes the claim under debate is dropped.

A fourth, implicit rule keeps the stance intact. Reported or conceded positions of the opposing side, such as “opponents argue X”, are not promoted to their own atom, because read in isolation they would carry the opposite stance of the speaker. Each argument is atomized independently, which allows parallelization. The resulting set of atomic arguments is denoted $\mathcal{A}'$ and forms the input to the next step. Every atom keeps the identifier of its source argument with a unique suffix, so that for instance pro5 can yield the atoms pro5 and pro5b, which share the base pro5.

4.3 Step 2: Attack Detection and Classification

With the atomic arguments in place, the pipeline examines every pair in $\mathcal{A}'$ for a potential attack, regardless of stance, since same-stance arguments can also conflict. Two pro-nuclear arguments may, for example, prioritise incompatible safety strategies. Pairs of atoms that share a base are skipped, since they originate from the same source argument.

Rather than a binary decision, the pipeline prompts the LLM to detect and categorize each potential attack at once and to return a confidence score and a justification. Asking for the type rather than a mere yes or no forces the model to engage with the argumentative structure of the pair instead of pattern matching on opposite stances, and the score and justification make

each decision traceable and provide a routing signal for the next step. Each candidate carries four fields.

```

1 direction:  A_ATTACKS_B | B_ATTACKS_A | NONE
2 attack_type: rebuttal | undercutting | undermining | NONE
3 confidence: integer in [0, 100]
4 reason:     one-sentence justification

```

Lst. 4.1: Fields of an attack candidate.

The prompt provides the full definitions of the three attack types from Section 2.2 and enforces that if `direction` is `NONE` then `attack_type` is `NONE` and `confidence` is zero, which keeps the output internally consistent. Rebuttals are treated as symmetric, so the reverse edge is added automatically, as motivated in Section 2.2. All $\binom{|A'|}{2}$ pairs are grouped into batches of size b , which reduces the number of API calls from quadratic to $\binom{|A'|}{2}/b$. Pairs for which the model returns malformed output are retried individually.

This typed prompt is exactly the component whose value the ablation in Section 5.3 measures, by comparing it against a binary variant that omits the attack type.

4.4 Step 3: Confidence Filtering and Verification

Not all candidates are equally reliable. The confidence score, which is an indication of the model’s certainty rather than a calibrated probability, routes each candidate through a three-way decision with two thresholds θ_{acc} and θ_{ver} .

$$\text{decision}(c) = \begin{cases} \text{accept} & c \geq \theta_{\text{acc}} \\ \text{verify} & \theta_{\text{ver}} \leq c < \theta_{\text{acc}} \\ \text{reject} & c < \theta_{\text{ver}} \end{cases} \quad (4.1)$$

Borderline candidates in the verify region are re-examined by a dedicated verifier prompt that asks the model to confirm or reject the attack given the two arguments and the proposed type, but without access to the original confidence score or justification. Withholding both keeps the verifier from anchoring on the extractor’s judgment. Accepted and confirmed candidates pass to the next step, and rejected ones are discarded.

4.5 Step 4: AAF Conversion

Every accepted candidate is converted to a binary Dung fact $\text{att}(A, B)$, deliberately dropping the attack type from the formal semantics. Unlike gradual

QBAF semantics, which assign each argument a numerical strength, Dung semantics ask which arguments can be jointly accepted, a set-based rather than numerical question, so binary attack facts are the natural representation. The attack type, the confidence score, and the justification are not lost. They are retained as comments and metadata and remain available for analysis and error inspection.

4.6 Step 5: Solving under Dung Semantics

The framework is solved by an AAF solver implemented in Prolog under the three semantics of Section 2.1, using standard fixed-point and enumeration procedures [3]. The grounded extension is computed by iterating the characteristic function from the empty set in polynomial time. Computing preferred and stable extensions requires enumerating conflict-free sets, which is exponential in the worst case.

To keep this tractable the solver applies a *grounded-first reduction*. The grounded extension and the arguments it attacks are fixed before the search, so the exponential enumeration is restricted to the undecided arguments that remain. This reduction is key to the scalability of the pipeline. On the frameworks in the evaluation it brings the preferred and stable computation down to a fraction of a second per topic, where a naive enumeration times out. The solver is parametrised by the framework file and returns its results as JSON, which the Python driver reads back.

Evaluation

This chapter evaluates the pipeline. Section 5.1 describes the experimental setup. Section 5.2 reports the cross-topic results over the eight UKP topics (RQ1, RQ4). Section 5.3 compares typed against binary prompting (RQ2), and Section 5.4 evaluates the attack quality against a manual gold standard (RQ3). Section 5.5 works through a small case study, and Section 5.6 analyses the recurring error types.

5.1 Experimental Setup

The pipeline is evaluated in two settings. The first starts from argument sets of the UKP Sentential Argument Mining Corpus [14], which provides sentence-level stance annotations for eight controversial topics. Each sentence is labelled `Argument_for`, `Argument_against`, or `NoArgument`. The `NoArgument` sentences are discarded and the test splits are used. For each topic a balanced sample of pro and con arguments is drawn with a fixed random seed for reproducibility. The second setting uses LLM-generated arguments on the topic of allowing autonomous vehicles in public, with ten pro and ten con arguments. Since the generation prompt already constrains each argument to a single claim, atomization is skipped in that setting.

In both settings the extractor and the verifier call the open-weight model GPT-OSS-120B at temperature zero, with argument pairs processed in batches of $b = 10$. Candidates are accepted at confidence $c \geq \theta_{\text{acc}} = 70$, forwarded to the verifier at $50 = \theta_{\text{ver}} \leq c < 70$, and rejected below θ_{ver} . All runs are logged with their configuration, seed, and per-pair decisions, so that every number reported here can be traced back to a concrete run.

5.2 Cross-Topic Results

Table 5.1 reports, for each of the eight UKP topics, the number of atomic arguments, the number of queried pairs, the number of accepted attacks and their share among all pairs, whether the framework contains a cycle, the size of the grounded extension, and the numbers of preferred and stable extensions.

Tab. 5.1.: Cross-topic results over the eight UKP topics. “Acc” is the number of accepted attacks, that is the edges of the framework, and “Acc%” is their share among all queried pairs. “Cyc” indicates a cycle, “Grnd” is the size of the grounded extension, and #Pr and #St are the numbers of preferred and stable extensions.

Topic	Atoms	Pairs	Acc	Acc%	Cyc	Grnd	#Pr	#St
Abortion	25	272	20	7.4	yes	12	2	2
Cloning	16	118	34	28.8	yes	5	1	1
Death Penalty	18	148	47	31.8	yes	2	3	3
Gun Control	23	244	20	8.2	yes	12	3	3
Marijuana Legal.	21	199	10	5.0	yes	16	2	2
Minimum Wage	22	221	55	24.9	yes	6	3	3
Nuclear Energy	20	183	35	19.1	yes	10	3	3
School Uniforms	21	202	28	13.9	yes	14	1	1

Two observations stand out. First, every framework contains cycles, a direct consequence of the rebuttal symmetrization described in Section 2.2. Second, six of the eight topics admit several preferred extensions, and the number of stable extensions matches the number of preferred extensions in every case. Only Cloning and School Uniforms reduce to a single extension. These multiple positions are not asserted by a free generation. They are derived from the model’s own pairwise attack judgments through the fixed semantics, and each is conflict-free and admissible by construction. The acceptance rate varies widely, from 5.0 percent for Marijuana Legalization to 31.8 percent for Death Penalty, which reflects genuine differences in how directly the sampled arguments engage each other.

5.3 Typed versus Binary Prompting

This section measures whether the typed attack prompting of Section 4.3, which the paper advertises as its central contribution, actually produces different or more reliable attacks than plain binary prompting. A second extraction arm runs the identical atomized argument set $\mathcal{A}'$ through a binary prompt that asks only whether an attack exists, without the attack type and without the Pollock definitions. The two resulting attack sets are compared per topic by their Jaccard similarity, both on the raw directed edges before symmetrization and on the final edges, and both frameworks are solved to compare the numbers of grounded, preferred, and stable extensions.

This experiment is planned and not yet carried out. Results and the corresponding table will be added here.

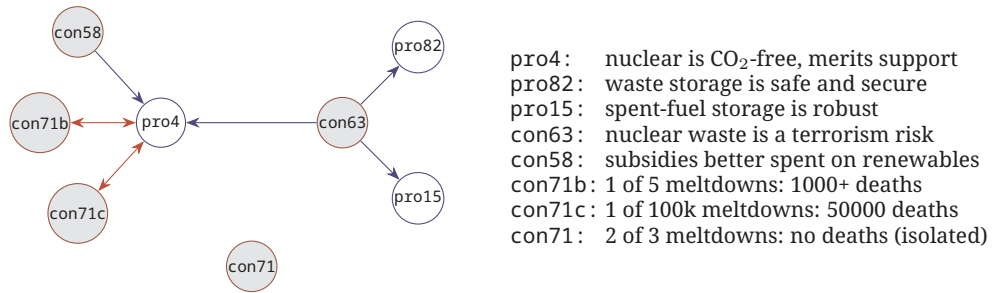


Fig. 5.1.: Attack graph of the small nuclear-energy sample, eight atoms and eight directed edges. Red double arrows are symmetric rebuttals, blue arrows denote undermining attacks. Filled nodes form the grounded extension, which here coincides with the unique preferred and stable extension.

5.4 Attack Quality against a Gold Standard

To move from feasibility to a measurable quality statement, one topic is annotated by hand at the level of argument pairs, and the pipeline's attack decisions are scored against this gold standard in terms of precision, recall, and F1. The result is placed next to the F1 around 67 percent reported by Kramer et al. [7] on the same corpus.

This experiment is planned and not yet carried out. Results will be added here.

5.5 Case Study: Nuclear Energy

For greater comprehensibility a small nuclear-energy sample is analysed in detail. Starting from six balanced UKP arguments, atomization yields eight atomic arguments, three pro and five con. Only the sentence about meltdown statistics is split, into three separate claims. Of the $\binom{8}{2} = 28$ pairs, 25 are queried after skipping pairs of atoms that share a source argument. Six attacks are accepted. Rebuttal symmetrization adds two reverse edges, so the framework has eight directed attacks, four undermining and four rebuttals.

The graph centers on the argument that nuclear power merits support because it is virtually CO₂-free, which the con side attacks from several directions. The symmetrized rebuttals make the framework cyclic, which suggests that several preferred and stable sets might exist. In this case, however, the grounded extension already terminates by accepting all five con arguments and rejecting the entire pro side, so grounded, preferred, and stable coincide on a single extension.

5.6 Error Analysis

The case study surfaces two recurring limitations, each from a different stage of the pipeline.

First, the attack-type classification can mislabel a premise-directed attack as a rebuttal. Since rebuttals are symmetrized into mutual attacks while undermining attacks stay directed, a false rebuttal label introduces a bidirectional conflict and therefore an unjustified cycle. The meltdown-fatality claims, for instance, attack the CO₂ argument as rebuttals, although they question its safety premise rather than directly negating its support conclusion. An undermining label would have been more suitable and would not have added a reverse edge.

Second, atomization can produce atoms that, read in isolation, no longer carry a clear stance and end up neither attacking nor attacked. The claim that two of three meltdowns cause no deaths, for example, understates rather than opposes the pro side on its own and therefore stays isolated in the graph.

Discussion

6.1 What the Formal Layer Adds

The cross-topic results in Section 5.2 show that the formal layer does uncover structure. Six of eight topics support several internally consistent positions, and every framework is cyclic. At the same time the value of the expensive preferred and stable enumeration over the cheap grounded extension is uneven. On several topics the grounded extension already decides most arguments, so preferred and stable add little, because the extraction tends to produce nearly acyclic graphs outside the symmetrized rebuttals. The clearest payoff appears on topics such as Death Penalty, Gun Control, and Minimum Wage, where three distinct stable extensions emerge. An honest reading is therefore that the formal layer is most useful where the debate genuinely splits into competing coherent camps, and closer to redundant where one side dominates.

6.2 Threats to Validity

Several threats qualify the results. The confidence score is a routing signal and not a calibrated probability, so the thresholds are design choices rather than learned values. In the LLM-generated setting the arguments, the attacks, and the confidence scores all come from the same model, so there is no independent quality signal, and that setting is treated as a qualitative check rather than a second evaluation. The UKP samples are small, with a balanced draw per topic, which limits how far the cross-topic numbers generalise. Finally, all results stem from a single model at temperature zero, and server-side execution does not guarantee perfect determinism.

6.3 Design Trade-offs

Three design decisions shape the frameworks and deserve explicit discussion. The symmetrization of rebuttals is well motivated by the absence of a preference ordering in Dung's abstract framework, but it is also the single source of every cycle, which makes a mislabelled rebuttal costly. Dropping the attack type when converting to binary attack facts is consistent with the set-based

question Dung semantics answer, yet it discards information that a richer formalism could use. The confidence thresholds trade precision against recall, and the verification step exists precisely to recover borderline cases without lowering the acceptance threshold for everyone.

Conclusion

This thesis presented an end-to-end pipeline that turns natural-language debate text into a formally evaluated Dung argumentation framework, from categorized attack extraction through confidence filtering to a Prolog solver under grounded, preferred, and stable semantics.

Returning to the research questions, RQ1 is answered in the affirmative. The pipeline runs end to end across eight topics and produces well-defined frameworks whose extensions can be computed in a fraction of a second per topic thanks to the grounded-first reduction. RQ4 is answered by the cross-topic analysis. All frameworks are cyclic, and six of eight admit several preferred and stable extensions, so the formal layer can make explicit several internally consistent positions that the raw argument set leaves implicit, although its advantage over the grounded extension is uneven across topics. The answers to RQ2 and RQ3 rest on the typed-versus-binary ablation and the gold-standard evaluation, which are set up in Chapter 5 and will be completed before submission.

Overall the results demonstrate the feasibility of integrating modern language models with established formal argumentation, and they suggest that formal semantics can provide a transparent reasoning layer on top of LLM-generated argumentative content.

7.1 Future Work

Several directions remain open. The extraction quality should be evaluated against annotated subtask datasets beyond the single gold standard used here. The typed attacks and confidence scores, which the current pipeline discards when converting to binary facts, could be used directly by a richer formalism such as ASPIC+ or a quantitative bipolar framework. Finally, the pipeline should be compared qualitatively against other methods, including a standalone LLM asked to summarise the positions in a debate, and across several models rather than a single one.

Bibliography

- [1] Gianvito Alfano et al. *LLM-based Argument Mining meets Argumentation and Description Logics*. 2026. arXiv: 2603.02858 (cit. on p. 10).
- [2] Ettore Caputo, Sergio Greco, and Lucio La Cava. “ARGUS: Towards End-to-End Argument Mining with Large Language Models”. In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 40. 48. 2026, pp. 41547–41549 (cit. on p. 9).
- [3] Federico Cerutti, Sarah Alice Gaggl, Matthias Thimm, and Johannes Peter Wallner. “Foundations of implementations for formal argumentation”. In: *Handbook of Formal Argumentation*. College Publications, 2014, pp. 319–383 (cit. on p. 16).
- [4] Phan Minh Dung. “On the acceptability of arguments and its fundamental role in nonmonotonic reasoning, logic programming and n-person games”. In: *Artificial Intelligence* 77.2 (1995), pp. 321–357 (cit. on pp. 1, 5).
- [5] Gabriel Freedman, Adam Dejl, Deniz Gorur, et al. “Argumentative large language models for explainable and contestable claim verification”. In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 39. 14. 2025, pp. 14930–14939 (cit. on p. 9).
- [6] Deniz Gorur, Antonio Rago, and Francesca Toni. “Can Large Language Models perform Relation-based Argument Mining?” In: *Proceedings of the 31st International Conference on Computational Linguistics*. Abu Dhabi, UAE: Association for Computational Linguistics, Jan. 2025, pp. 8518–8534 (cit. on p. 10).
- [7] Stefan Kramer et al. *Focusing knowledge-based graph argument mining via topic modeling*. 2021. arXiv: 2102.02086 (cit. on pp. 10, 19).
- [8] Paul Krause, Simon Ambler, Morten Elvang-Goransson, and John Fox. “A logic of argumentation for reasoning under uncertainty”. In: *Computational Intelligence* 11.1 (1995), pp. 113–131 (cit. on p. 1).
- [9] John Lawrence and Chris Reed. “Argument mining: A survey”. In: *Computational linguistics* 45.4 (2019), pp. 765–818 (cit. on p. 7).
- [10] Hao Li, Viktor Schlegel, Yizheng Sun, Riza Batista-Navarro, and Goran Nenadic. “Large language models in argument mining: A survey”. In: *arXiv preprint arXiv:2506.16383* (2025) (cit. on p. 7).
- [11] Sanjay Modgil and Henry Prakken. “The ASPIC⁺ framework for structured argumentation: A tutorial”. In: *Argument & Computation* 5.1 (2014), pp. 31–62 (cit. on pp. 6, 7, 11).

- [12]Paulo Pirozelli, Victor Hugo Nascimento Rocha, Fabio G. Cozman, and Douglas Aldred. *An LLM-Based System for Argument Mining*. 2026. arXiv: 2605.13793 [cs.CL] (cit. on p. 9).
- [13]John L. Pollock. “How to reason defeasibly”. In: *Artificial Intelligence* 57.1 (1992), pp. 1–42 (cit. on pp. 6, 10).
- [14]Christian Stab, Tristan Miller, Benjamin Schiller, Pranav Rai, and Iryna Gurevych. “Cross-topic Argument Mining from Heterogeneous Sources”. In: *Proceedings of EMNLP 2018*. 2018, pp. 3664–3674 (cit. on pp. 2, 10, 17).

List of Figures

2.1. Running example of a small argumentation framework. To be replaced with a TikZ figure and a step-by-step computation of the grounded, preferred, and stable extensions.	6
4.1. Overview of the pipeline, from raw debate text to formally evaluated Dung extensions in five steps.	14
5.1. Attack graph of the small nuclear-energy sample, eight atoms and eight directed edges. Red double arrows are symmetric rebuttals, blue arrows denote undermining attacks. Filled nodes form the grounded extension, which here coincides with the unique preferred and stable extension.	19

List of Tables

5.1. Cross-topic results over the eight UKP topics. “Acc” is the number of accepted attacks, that is the edges of the framework, and “Acc%” is their share among all queried pairs. “Cyc” indicates a cycle, “Grnd” is the size of the grounded extension, and #Pr and #St are the numbers of preferred and stable extensions. 18

A.1. This is a caption text. 33

A.2. This is a caption text. 34

List of Listings

4.1. Fields of an attack candidate.	15
---	----

Example Appendix

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

A.1 Appendix Section 1

This is the second paragraph. Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Tab. A.1.: This is a caption text.

Alpha	Beta	Gamma
0	1	2
3	4	5

A.2 Appendix Section 2

And after the second paragraph follows the third paragraph. Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives

you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Tab. A.2.: This is a caption text.

Alpha	Beta	Gamma
0	1	2
3	4	5

After this fourth paragraph, we start a new paragraph sequence. Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language. Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Colophon

This thesis was typeset with $\text{\LaTeX} 2_{\epsilon}$. It uses the *Clean Thesis* style developed by Ricardo Langner. The design of the *Clean Thesis* style is inspired by user guide documents from Apple Inc.

Download the *Clean Thesis* style at <http://cleanthesis.der-ric.de/>.

Adaptations to the style of the Institute of Computer Science can be found at <https://gitlab.rlp.net/institut-fur-informatik/cleanthesis-jgu>.

Eigenständigkeitserklärung

Hiermit erkläre ich, dass ich die vorliegende Arbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe (dazu zählen auch „KI-Werkzeuge“). Sämtliche wörtlichen und sinngemäßen Übernahmen und Zitate sind kenntlich gemacht und nachgewiesen. Auch beim Einsatz von KI-Werkzeugen¹ stellt die Arbeit klar dar, in welchem Umfang diese genutzt wurden und welche Inhalte dadurch erzeugt oder beeinflusst wurden². Ich versichere, dass ich keine Hilfsmittel verwendet habe, deren Nutzung die Prüferinnen und Prüfer explizit ausgeschlossen haben.

Mit Abgabe der vorliegenden Leistung übernehme ich die Verantwortung für das eingereichte Gesamtprodukt. Ich verantworte damit auch alle KI-generierten Inhalte, die ich in meine Arbeit übernommen habe. Die Richtigkeit übernommener (KI-generierter) Aussagen und Inhalte habe ich nach bestem Wissen und Gewissen geprüft.

Ich habe die Arbeit nicht zum Erwerb eines anderen Leistungsnachweises in gleicher oder ähnlicher Form eingereicht. Von der Ordnung zur Sicherung guter wissenschaftlicher Praxis und zum Umgang mit wissenschaftlichem Fehlverhalten habe ich Kenntnis genommen.

Mir ist bekannt, dass ein Verstoß gegen die genannten Punkte prüfungsrechtliche Konsequenzen hat und insbesondere dazu führen kann, dass die Studien- und Prüfungsleistung als mit „nicht bestanden“ bewertet wird. Die Einschreibung kann für bis zu zwei Jahre widerrufen werden, wenn Studierende zweimal oder häufiger bei Prüfungsleistungen täuschen (§ 69 Abs. 4 und 5 HochSchG).

Mainz, July 2, 2026

Aaron Habyarimana

¹Mit „KI-Werkzeugen“ werden computergestützte Systeme bezeichnet, die auf Basis maschinellen Lernens neue Inhalte generieren können (z. B. ChatGPT, Gemini, Claude, LLaMA, Midjourney, Stable Diffusion o. ä.).

²Die Nutzung von KI-Werkzeugen muss dann kenntlich gemacht werden, wenn Sie den Kern der Aufgabenstellung in von den Prüfern nicht anzunehmender Weise berührt: Sollte nichts anderes explizit mit den Prüferinnen und Prüfern vereinbart worden sein, so ist dies der Fall, sobald wesentliche Teile der eingereichten Arbeit (z. B. ganze Sätze des Textes, ganze Abbildungen, nicht-offensichtliche Teile von Rechnungen oder Beweisen, mehrere Zeilen von Programmcode am Stück) von solchen Werkzeugen generiert wurden und wörtlich oder in abgewandelter Form in die Arbeit übernommen wurden oder wesentliche Konzepte oder Ideen (z. B. eine Gliederung der Literatur oder ein Lösungsansatz für ein Problem) von KI-Werkzeugen generiert wurden.

Hinweis: Es wird im Falle von abweichenden Vereinbarungen mit den Prüferinnen und Prüfern empfohlen, jene im Vorfeld schriftlich festzuhalten und zusätzlich auch noch einmal in der eingereichten Arbeit zu benennen.

Use of AI tools.

AI Tool	Used for	Reason	When

